■ 30-Day Python Course

Day 03 — Control Flow

Phase: Beginner · Duration: 2 Hours · Intermediate to if/elif/else, match

■ **Day 3 of 30**	■■ **2 Hours**	■ **Beginner Phase**	■ **5 Concepts**

Learning Objectives

- ■ Write conditional logic using **if / elif / else** to control program flow
- ■ Apply all six **comparison operators** and three **logical operators**
- ■ Build **nested conditions** for complex decision trees
- ■ Use Python 3.10+ **match / case** structural pattern matching
- ■ Recognise and avoid common conditional pitfalls (equality vs identity, truthy/falsy)

Prerequisites

Requirement	Details
Day 1 complete	Python 3.12, VS Code, uv environment working
Day 2 complete	Variables, data types, int/float/str/bool, type casting
Variables comfort	Can create and print variables; understands type()

■ Section 2 — Session Overview

Session Goal

By the end of this session students will write Python programs that make decisions — branching into different paths based on conditions — using if/elif/else, logical operators, nested conditions, and the modern match statement.

Minute-by-Minute Timeline

Time	Activity	Notes
0:00–0:10	Warm-up recap quiz — Day 2 variables	3 quick-fire questions
0:10–0:25	Concept 1 — if / elif / else syntax	Live code: grade checker
0:25–0:40	Concept 2 — Comparison & logical operators	Truth table demo
0:40–0:55	Concept 3 — Nested conditions	Live code: login system
0:55–1:05	Break	
1:05–1:25	Concept 4 — match / case statement	Python 3.10+ only
1:25–1:40	Concept 5 — Truthy / Falsy values	Common pitfall prevention
1:40–2:00	Mini-project: CLI menu system	Students code independently

■ **Instructor note:** Resist showing match/case too early — students need to be comfortable with if/elif first. match/case is syntactic sugar, not a replacement for understanding boolean logic.

■ Section 3 — Pre-Session Checklist

- ■ Virtual environment activated (**uv** or venv)
- ■ VS Code open with a new file: **day03_control_flow.py**
- ■ Python interpreter set to 3.12 in VS Code status bar
- ■ Ruff linter active (yellow squiggles on bad indentation)
- ■ Day 2 recap: can run **print(type(42))** without error
- ■ Slides/screen share ready (if recording)

■■ **Windows users:** Ensure they are running Python from the activated venv, not the system Python. Check: **python --version** should print **3.12.x**.

■ Section 4 — Step-by-Step Instruction Guide

Step 1 — if / elif / else Syntax (15 min)

■ What to Say:

"Every program needs to make decisions. In Python the way we do that is with the if statement. Think of it like a fork in the road — Python evaluates a condition and then takes one of two (or more) paths."

Syntax Breakdown:

```
# Basic if / elif / else structure
if condition:
# runs when condition is True
elif another_condition:
# runs when first condition is False, this one is True
else:
# runs when ALL conditions above are False
```

Live Coding Example — Grade Checker:

```
# day03_control_flow.py — Grade checker
score = 72

if score >= 90:
grade = "A"
elif score >= 80:
grade = "B"
elif score >= 70:
grade = "C"
elif score >= 60:
grade = "D"
else:
grade = "F"

print(f"Score: {score} → Grade: {grade}")
# Output: Score: 72 → Grade: C
```

Common Error	Cause	Fix
IndentationError	Mixing tabs and spaces	Use 4 spaces, enable 'Render Whitespace' in VS Code
SyntaxError: expected ':'	Missing colon after condition	Always end if/elif/else with a colon
elif after else	else must be last	Reorder: if → elif → else

■ **Done when:** Students can change **score** to different values and get the correct grade each time.

Step 2 — Comparison & Logical Operators (15 min)

■ What to Say:

"Conditions are built from operators. Python gives us six comparison operators and three logical operators. Let's build a truth table live so you can see exactly how they behave."

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	7 > 4	True
<	Less than	2 < 9	True
>=	Greater or equal	5 >= 5	True
<=	Less or equal	3 <= 7	True

```
# Logical operators: and, or, not
age = 25
has_id = True

if age >= 18 and has_id:
print("Entry allowed")

if age < 13 or age > 65:
print("Discounted ticket")

if not has_id:
print("ID required")
```

■■ **Pitfall: ==** checks equality, **=** assigns a value. Writing **if age = 18:** is a SyntaxError in Python (unlike some other languages).

Step 3 — Nested Conditions (15 min)

■ What to Say:

"Sometimes a condition depends on another condition. We can put an if inside another if — this is called nesting. Be careful: too many levels of nesting is a code smell. Two levels is usually fine, three is the limit."

Live Coding Example — Login System:

```
# Simple login system with nested conditions
username = "admin"
password = "secret123"

if username == "admin":
if password == "secret123":
print("Welcome, admin!")
else:
print("Wrong password.")
else:
print("User not found.")
```

Refactoring Nested → Flat with 'and':

```
# Often cleaner to combine with 'and'
if username == "admin" and password == "secret123":
print("Welcome, admin!")
else:
print("Invalid credentials.")
```

■ **Done when:** Students can explain why flat conditions with **and** are often preferred over deeply nested ifs.

Step 4 — match / case Statement (20 min)

■ What to Say:

"Python 3.10 introduced structural pattern matching with the match statement. If you've used switch/case in other languages, this will feel familiar — but Python's version is much more powerful. Today we'll cover the basics."

```
# match/case — Python 3.10+ only
command = "quit"

match command:
case "start":
print("Starting...")
```

```
case "stop":
print("Stopping...")
case "quit":
print("Goodbye!")
case _: # wildcard — matches anything
print("Unknown command")
```

Matching Multiple Values in One Case:

```
# Use | to match multiple values
day = "Saturday"
match day:
case "Saturday" | "Sunday":
print("Weekend!")
case "Monday" | "Friday":
print("Almost there!")
case _:
print("Midweek grind")
```

■ **Version check:** match/case requires Python ≥ 3.10. Run **python --version** to confirm. On older versions, use if/elif chains instead.

Step 5 — Truthy / Falsy Values (15 min)

■ What to Say:

"In Python, every value has a boolean interpretation — it's either truthy or falsy. This means you can use values directly in if statements without comparing them explicitly. This is idiomatic Python and you will see it everywhere."

Falsy Values	Truthy (everything else)
False	True
None	Any non-zero number
0 (int zero)	Any non-empty string: "hello"
0.0 (float zero)	Non-empty list, dict, set, tuple
"" (empty string)	Any object instance (default)
[] {} () set()	

```
# Truthy/Falsy in practice
name = ""

if name: # falsy if empty string
print(f"Hello, {name}")
else:
print("Name is empty!")

# Checking for None explicitly
value = None
if value is None: # use 'is', not '=='
print("No value provided")
```

■■ **Identity vs equality:** Use **is None** and **is not None** to check for None. Use **== None** is technically correct but **is** is the Python idiom and faster.

■ Section 5 — Session Wrap-Up

Student Checklist

- ■ I can write if / elif / else blocks with correct indentation
- ■ I know all six comparison operators and all three logical operators
- ■ I can use **and** / **or** / **not** to combine conditions
- ■ I understand the difference between **==** and **=**
- ■ I can write nested conditions and know when to flatten them
- ■ I have run at least one example using match / case
- ■ I understand truthy and falsy values and can use bare values in if statements
- ■ I know to use **is None** not **== None**

Key Concepts Recap

if / elif / else	The core branching construct. Only one branch executes.
Comparison operators	== != > < >= <= — always return True or False.
Logical operators	and / or / not — combine or invert boolean expressions.
Nested conditions	if inside if — limit nesting to 2 levels; prefer 'and'.
match / case	Python 3.10+ pattern matching. Use | for multiple values. _ as wildcard.
Truthy / Falsy	0, None, empty strings/collections are falsy. Everything else truthy.

■ **Next up — Day 4: Loops.** Students will combine today's conditions with for and while loops to repeat code blocks. Encourage them to complete the Day 3 practice problems before the next session.

■ Section 6 — Quick Reference Card

Control Flow Syntax Cheat Sheet

```
# ■■ if / elif / else ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
if condition:
    ...
elif other:
    ...
else:
    ...

# ■■ Comparison operators ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
== != > < >= <=

# ■■ Logical operators ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
and or not

# ■■ match / case (3.10+) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
match value:
    case "a" | "b":
        ...
    case _:
        ...

# ■■ Falsy values ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
False None 0 0.0 '' [] {} () set()

# ■■ Identity checks ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
if value is None: ...
if value is not None: ...
```

Operator Precedence (highest → lowest)

Priority	Operator	Notes
1 (highest)	Comparison operators == != > < >= <=	Evaluated before logical
2	not	Inverts a single boolean
3	and	Both sides must be True
4 (lowest)	or	Either side must be True

VS Code Shortcuts

Ctrl/Cmd + /	Toggle line comment
Ctrl/Cmd + D	Select next occurrence of selected word
Alt + Up/Down	Move line up/down
Ctrl/Cmd +]	Indent selected block
Ctrl/Cmd + [	Outdent selected block
F5	Run/Debug current Python file